

嵌入式第五次作业

姓名：李丰克 学号：3230105182

1 问题描述

做一个控制系统，输入是温度/光敏传感器，输出是步进/直流电机。

设定一个初始值（以当前温度/亮度为参考），如果传感器温度/亮度超过初始值，则电机正转；如果低于设定值，则电机反转；当前值和初始值差异越大，则电机转速越快

2 问题分析

本次作业我的传感器选择光敏传感器，输出为步进电机。

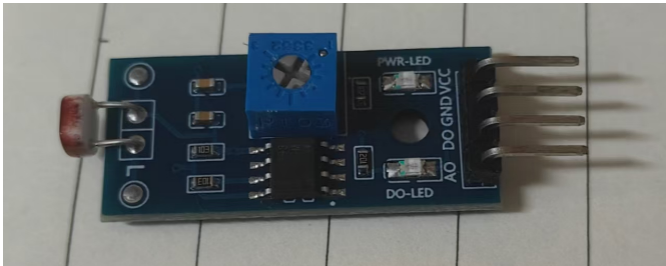
预期结果可以描述为：光照光敏电阻变化，在传感器上输出模拟量，经过adc模数转换器转换为数字量，数字量再映射成电压值，再根据传感器的电路反推出光敏电阻值，由光敏电阻值映射到光照强度；将光照强度显示在lcd屏幕上，lcd共显示两个数据，一个为基准光照强度，一个为当前光照强度；通过按下按键K1可以将基准光照强度更新为当前的光照强度；电机选择为步进电机，限制其转速在一个区间[0,max]内，当光照强度小于基准值反转，大于基准值正转。将当前光照强度与基准值的差线性映射到速度区间上，实现偏离基准值越多转速越快，更详细的分配策略见后续。

分析问题的要求，可以将其分解为各个小模块：光敏传感器获取光照强度，ADC模数转换，LCD屏幕显示，按键控制，步进电机控制。

3 模块实现

3.1 光照强度的获取

3.1.1 光敏传感器



此为本次用到的光敏传感器，其光敏电阻型号为GL5516，共有四个管脚，依次为：

VCC	3.3V恒压
GND	接地
AO	输出模拟量，接到PA3管脚转换为数字量

3.1.2 数模转换

adc的管脚定义为PA3，管脚的定义以及adc相关的配置忽略

```
u16 Get_ADC_Value(u8 ch,u8 times)
{
    //times 是获取的次数，依照此次数取平均值
```

```

u32 temp_val=0;
u8 t;
ADC_RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5);

for(t=0;t<times;t++)
{
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);
    while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC ));
    temp_val+=ADC_GetConversionValue(ADC1);
    delay_ms(5);
}
return temp_val/times;
}

```

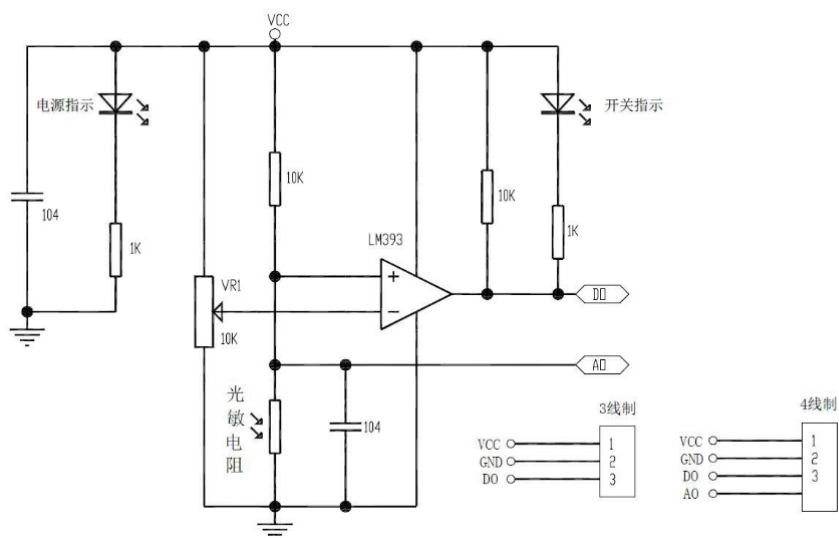
此时通过串口调试器，可以读出此管脚接收到的数字量。因为光敏传感器恒压源是3.3V，那么AO管脚输出的最大电压为3.3V，令其电压最大得到的adc分辨率为4096，根据(ADC分辨率 / 参考电压) = (ADC数字值 / 实际电压值)可以将得到的数字量转化为实际电压值。

```

vol=(float)value*(3.3/4096);

```

3.1.3 光照强度的计算



由管脚可以直接读出电压值，光敏传感器的电路图如上所示，由有AO输出的那一线路可以看出，光敏电阻与10KΩ的电阻分压。

$$V_{AO} = 3.3 \times \frac{R_{光}}{10 + R_{光}}$$
，R光的单位为千欧。由此可以将电压值转化为电阻值。

```

R0=10*vol/(3.3-vol);

```

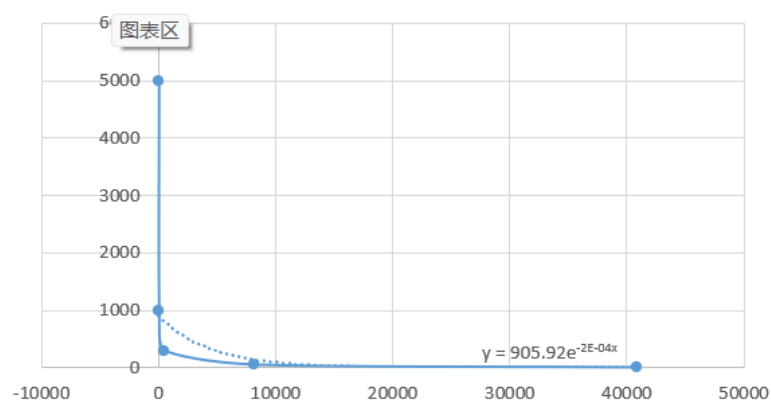
```

guangqiang: 25 lux
检测AD值为: 1000
检测电压值为: 0.81V
guangmindianzu: 3.2300K
guangqiang: 95 lux
检测AD值为: 433
检测电压值为: 0.35V
guangmindianzu: 1.1821K
guangqiang: 715 lux
检测AD值为: 229
检测电压值为: 0.18V
guangmindianzu: 0.5922K
guangqiang: 2851 lux
检测AD值为: 217
检测电压值为: 0.17V
guangmindianzu: 0.5594K
guangqiang: 3195 lux

```

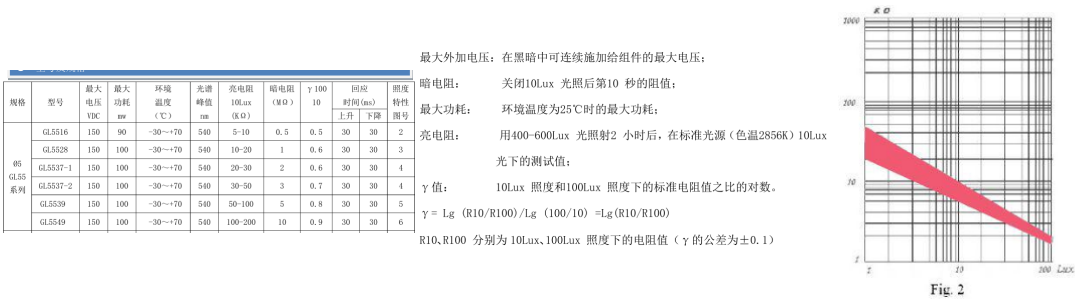
但是关于光敏电阻阻值与光照强度的映射关系，我在网络上没有搜索到一个确切的公式，均与光敏电阻的材料等有关。又因为我没有可以测光照强度的仪器，我做出了如下近似估计

1		R	I/lux
2	手机手电筒2cm直射	0.276	5000
3	阴天日光灯管50cm直射	15.1228	1000
4	距离手机屏幕5cm	471.882	300
5	阴天室内关灯距离窗户3m	8181.9033	50
6	手完全盖住	40900	0.1



其中阻值是根据串口调试器直接读出来的，光照强度是根据事件加上AI估计出来的，拟合成一个非常非线性的图象，由图我们可以看出，光照强度趋于0，阻值趋于无穷；光照强度趋于无穷，阻值趋于0；在日常时光照强度大约为200-400，此时阻值在10K左右。excel给出的趋势线函数肯定是错的，因为不满足光照强度小于零阻值无穷大。

前面已经提到光敏电阻的型号为GL5516，于是我从光敏电阻的规格书入手。



由第一个表格可以得出相关参数，从中读出γ参数为0.5，但是对于其亮电阻10lux--10kΩ表示存疑，因为我在实验中得出应该为400lux左右。再参考第二张图对这些参数的解释，大致可以得出此亮电阻应该不是我理解的亮电阻。再由第三章图可以看出光照强度的对数lgI与阻值的对数lgR有着负相关的线性关系，即 $\frac{\lg R - \lg R_0}{\lg I - \lg I_0} = -k$ ，我们再结合γ参数的计算公式，恰是刚才线性关系的一种特殊情况，因此我们可以令k为γ参数，参考值我们可以定为400lux--10KΩ，于是我们有：

```
lux=400*pow((10/R0),2);
```

对于这个拟合公式，在光照强度较小的时候，还是比较符合的；但是当光照强度过大时，比如那手机手电筒直射光敏电阻，此时示数能达上万，此时有些发散了，但是近似上，在日常应用场景里已经差不多了。

3.2 LCD1602显示

3.2.1 LCD1602管脚

LCD1602共有16个管脚，其中工作管脚有11个，另外的AK管脚分别接5V恒压与地，给背光板供电；左侧的VSS，VDD分别接地和5V恒压，给液晶供电；此外还有一个V0管脚，用于调节对比度的，若过高，液晶显示变浅，甚至消失；若过低，显示的液晶与非显示的对比度不够，也不好看出。如果将V0接地的话就会出现后者的情况，因此**V0应该接一个较小但不为0的电压，最好还可以调节。但是开发板上并没有这种引脚**，于是我用一根公对母杜邦线将其接到了开发板自带的lcd接口管脚的V0处。

RS	PA8
RW	PB6
E	PB7
D0~D7	PB8~PB15

3.2.2 LCD显示

lcd配置函数忽略，以下为其主要函数

```
void LCD1602_waitReady(void)//用于LCD忙检测
{
    u8 sta = 0;
    LCD1602_GPIO_Init_Out();//LCD1602端口初始化,配置为输出
    GPIOB->ODR =0xff00;
    LCD1602_RS_RESET();
    LCD1602_RW_SET();//读状态
    LCD1602_GPIO_Init_Inupt();//LCD1602端口初始化,配置为输入
    do{
        LCD1602_E_SET();
        sta = GPIO_ReadInputDataBit(LCD1602_GPIOB, GPIO_Pin_15);
        LCD1602_E_RESET();
    }
    while(sta);
    LCD1602_GPIO_Init_Out();//LCD1602端口初始化,配置为输出
}
void LCD1602_writeCmd(u16 cmd)//用于写指令
{
    LCD1602_waitReady();//等待液晶准备好
    LCD1602_RS_RESET();
    LCD1602_RW_RESET();//写指令

    GPIOB->ODR &= 0x00FF;
    GPIOB->ODR |= (cmd << 8);
    LCD1602_E_SET();
    delay_us(5);
    LCD1602_E_RESET();
    delay_us(5);
}
void LCD1602_writeDate(u16 date)//用于写数据
{
    LCD1602_waitReady();//等待液晶准备好
    LCD1602_RS_SET();
```

```

LCD1602_RW_RESET();//写数据

GPIOB->ODR &= 0x00FF;
GPIOB->ODR |= (date << 8);
LCD1602_E_SET();
delay_us(5);
LCD1602_E_RESET();
delay_us(5);
}
void LCD1620_SetAddress(unsigned char x,unsigned char y)
{
    if(y == 0)
        LCD1602_writeCmd(0x80 | x);//从第一行开始显示
    else
        LCD1602_writeCmd(0x80 | 0x40 | x);//从第二行开始显示
}
void LCD1602_ShowStr(unsigned char x,unsigned char y,unsigned char *str)//LCD1602
显示字符串
{
    LCD1620_SetAddress(x,y);//设置数据地址指针
    while(*str != '\0')
        LCD1602_writeDate(*str++);//写数据
}
void LCD1602_Init(void)//液晶初始化函数
{
    LCD1602_GPIO_Init_Out();
    delay_ms(15); //上电延时
    LCD1602_writeCmd(0x38);
    delay_ms(5);
    LCD1602_writeCmd(0x38);
    delay_ms(5);
    LCD1602_writeCmd(0x38);
    delay_ms(5);
    LCD1602_writeCmd(0x0c);
    delay_ms(5);
    LCD1602_writeCmd(0x06);
    delay_ms(5);
    LCD1602_writeCmd(0x01);
    delay_ms(5);
}

```

在主函数初始化后，便可以直接将显示的字符串用LCD1602_ShowStr()函数显示了。

```

data1[0]=tool[lux_based/10000];
data1[1]=tool[(lux_based/1000)%10];
data1[2]=tool[(lux_based/100)%10];
data1[3]=tool[(lux_based/10)%10];
data1[4]=tool[lux_based%10];

```

其中显示的光照强度都是整数便于处理，为了方便转化为字符串，此处[参考了数码管的操作](#)，即定义一个工具字符串tool={"0123456789"}，存有每个数字的字符形式，且数字与下标对应，这样编可以直接调用，将整数转为字符串。

以下为显示效果，左为基准值，右为当前光照强度。



3.3 按键

按键功能比较简单，只要定义一个引脚，添加一个按键监测函数即可。

```
u8 KEY_Scan(void)
{
    static u8 key=1;
    if(key==1&&KEY1==0)
    {
        delay_ms(10);
        key=0;
        return 1;
    }
    else if(KEY1==1)
    {
        key=1;
    }
    return 0;
}
```

在主函数中再添加

```
key_value=KEY_Scan();
if (key_value==1){
    lux_based=lux;
    key_value=0;
}
```

即可实现，当按键按下时，基准值更新为当前光照强度。

3.4 电机

3.4.1 电机的选择

为什么不选择直流电机呢，如果选择直流电机，用pwm波即可调转速，这样将与基准值的差映射为占空比即可完成差值与转速的对应。但是非常致命的一点是，直流电机不能控制正反转（至少我在仅用这块板子的资源的基础上实现不了）。

直流电机只有两个输入端口，其中一个固定为5V横流源，另外一个为pwm波输入控制转速。而使其反转的方法本质上只能让另一个端口输入5V，这一个输入pwm波。比较典型的解决方案有**H桥**，**双继电器**等等。对于H桥，我没有；继电器我只有一个，而且我还需要双头的杜邦线来实现并联。开发板上关于电机的驱动有两块芯片——ULN2003和TC1508S，理论上TC1508S是可以支持直流电机反转的，**但是开发板只引出了一半的引脚**，导致不能实现了。此外我还想过，能否两端都输入pwm波，令其中一路的占空比为100%，能否看作直流恒压。理论上是可行的，但是实际上，即使其占空比是100%，其本质上也是个谐波，含有谐波分量，因此直流电机不可行。

3.4.2 步进电机

实验箱配备的为四线四相电机，用TC1508S芯片驱动，原理为依次给四端的电磁铁供电，使电机可以转动，根据步进角的不同可以分成不同拍，如四拍和八拍。

A+	OA--IA	PA12
A-	OB--IB	PA0
B+	OC--IC	PA5
B-	OD--ID	PA7

转动原理如下，使用的是四拍

```
u8 reverse[4] = {0x03,0x09,0x0c,0x06}; //反转
u8 forward[4]= {0x03,0x06,0x0c,0x09}; // 正转
void SetMotor(unsigned char InputData)
{
    if(InputData == 0x03)
    {
        GPIO_SetBits(GPIOA,GPIO_Pin_12);
        GPIO_ResetBits(GPIOA,GPIO_Pin_0);
        GPIO_SetBits(GPIOA,GPIO_Pin_5);
        GPIO_SetBits(GPIOA,GPIO_Pin_7);
    }
    else if(InputData == 0x09)
    {
        GPIO_SetBits(GPIOA,GPIO_Pin_12);
        GPIO_SetBits(GPIOA,GPIO_Pin_0);
        GPIO_SetBits(GPIOA,GPIO_Pin_5);
        GPIO_ResetBits(GPIOA,GPIO_Pin_7);
    }
    else if(InputData == 0x0c)
    {
        GPIO_ResetBits(GPIOA,GPIO_Pin_12);
        GPIO_SetBits(GPIOA,GPIO_Pin_0);
        GPIO_SetBits(GPIOA,GPIO_Pin_5);
        GPIO_SetBits(GPIOA,GPIO_Pin_7);
    }
}
```

```

}
else if(InputData == 0x06)
{
    GPIO_SetBits(GPIOA,GPIO_Pin_12);
    GPIO_SetBits(GPIOA,GPIO_Pin_0);
    GPIO_ResetBits(GPIOA,GPIO_Pin_5);
    GPIO_SetBits(GPIOA,GPIO_Pin_7);
}
else if(InputData == 0x00)
{
    GPIO_ResetBits(GPIOA,GPIO_Pin_12);
    GPIO_ResetBits(GPIOA,GPIO_Pin_0);
    GPIO_ResetBits(GPIOA,GPIO_Pin_5);
    GPIO_ResetBits(GPIOA,GPIO_Pin_7);
}
}

void motor_circle(int n, int direction, int delay)
{
    int i, j;
    if(n==0){SetMotor(0x00);}
    else{
        for(i = 0; i < n ; i++)
        {
            for(j = 0; j < 4; j++)
            {
                if(1 == direction)
                {
                    SetMotor(forward[j]);
                }
                else
                {
                    SetMotor(reverse[j]);
                }
                delay_ms(delay > 2 ? delay : 2);
            }
        }
    }
}
}

```

3.4.3 步进电机的转速控制

步进电机的程序在主程序中运行时，如果用motor_circle()函数一直加大圈数，会发现其转速一段时间后不会再提高，在圈数较小时，电机转完所需的时间远小于主循环中其他数据处理的时间；当提高到一定数量后，电机转动成为了主要消耗时间的任务，这就导致**在若干个while循环中，转过圈数的总和除以时间越来越趋近于其转一圈所用的固定时间，即到达了最大值**，不能再增，再增加徒增时间消耗。因此为了实验现象明显，我们将电机的转速设定在一个小区间内，最大值约为一个while循环中120圈。

3.5 主程序文件

3.5.1 差值与转速的映射

在实验过程中，我们发现光照强度是非常不线性的一个量，日常情况下为400lux左右，在光照强度较小时，从完全黑暗的0到400lux的变化速度远小于从400lux升高的速度。而且光照强度基准值较难达到一千，因此做出以下映射策略；

当光照强度在基准值上下不超过30lux范围内，电机不转动；

当光照强度小于基准值-30lux：

direction=-1，反转

如果基准值小于1030(1000+30)， $speed = max_speed * (lux_based - lux) / (lux_based - 30) + 1$ ；//其中+1是防止取整后为0

如果基准值大于1030：

当lux大于基准值-1030， $speed = (lux_based - lux) / 1000 * max_speed + 1$ ；

当lux小于基准值-1030， $speed = max_speed$ ，达到最大值。

当光照强度大于基准值+30lux：

direction=1，正转

当lux小于基准值+1030， $speed = (lux - lux_based) / 1000 * max_speed + 1$ ；

当lux大于基准值+1030， $speed = max_speed$ ，达到最大值。

上述的speed值对应的是圈数，因此会做取整处理。

3.5.2 各个模块的组合

如果单纯的将电机控制函数motor_circle (speed, direction, 2) 放在while的开头，那么当圈数开始上升的时候，电机的转动与数据的处理是非常割裂的，电机转动会有明显的卡顿感，为了解决这个问题，有三个方案。

3.5.2.1 中断处理实现伪多线程

确定电机转动与数据处理的优先级，将电机转动改为中断事件，每个一段时间触发一次，更改中断的间隔时长可以实现电机的转速控制。事实上当间隔时长非常小的时候，还是会影响数据处理的速度。再加上这个功能有些复杂，没有完成。

3.5.2.2 pwm波模拟步进电机输入

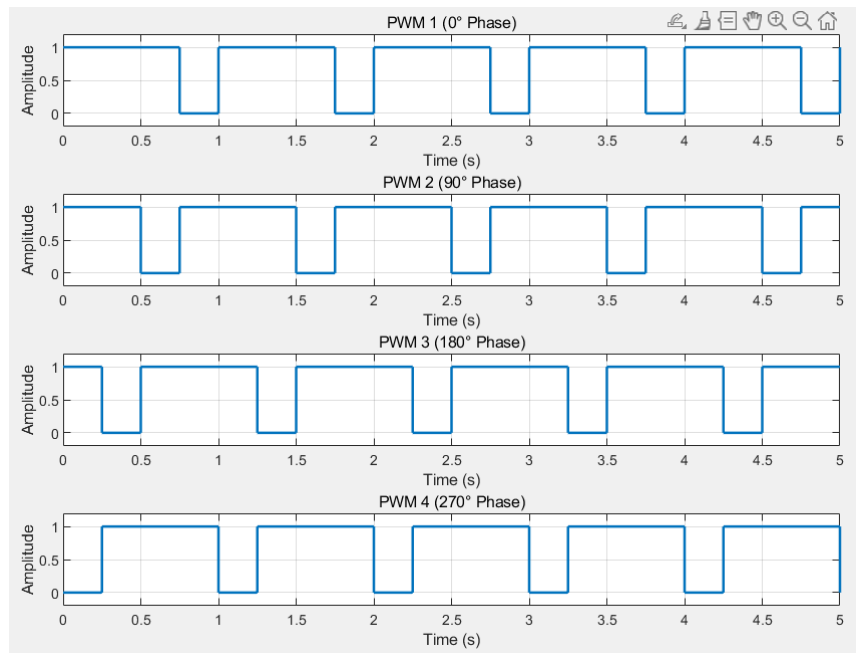
我们理想的情况是电机可以脱离主循环，自行转动，而主循环只需要控制电机开闭以及调节其参数即可。在不支持多线程的stm32f103c8芯片上，唯一能够实现类似功能的是pwm波，自初始化开始便不受主循环控制，一直在输出，而且还可以调节其参数。

再者，我们观察步进电机的四拍，令1为高电平

A+	A-	B+	B-
1	0	1	1
1	1	1	0
0	1	1	1

A+	A-	B+	B-
1	1	0	1

我们恰好可以将其看作四个通道的占空比为75%的，依次相邻相差90度相位(可以调顺序)的pwm波。



我选择定时器TIM3的四个通道，依次对应PA6,PA7,PB0,PB1四个管脚输出。定义为：

```
void TIM3_CH1_PWM_Init(u16 per,u16 psc)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_OCInitTypeDef TIM_OCInitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;

    /* ??? */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);

    /* ??GPIO (TIM3_CH1??PA6) */
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; // ?????
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    /* TIM3???? */
    TIM_TimeBaseInitStructure.TIM_Period = per; // ???(ARR)
    TIM_TimeBaseInitStructure.TIM_Prescaler = psc; // ???
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);

    /* ??1?: ??0111 */
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1; // PWM??1
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = QUARTER_CYCLE(per); // ?????1/4??,??0111??
```

```

    TIM_OC1Init(TIM3, &TIM_OCInitStructure);

    TIM_OC1PreloadConfig(TIM3, TIM_OCPreload_Enable);
    TIM_ARRPreloadConfig(TIM3, ENABLE);

    TIM_Cmd(TIM3, ENABLE);
}
void TIM3_CH2_PWM_Init(u16 per, u16 psc)
{
    ...
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = QUARTER_CYCLE(per) * 2;
    TIM_OC3Init(TIM3, &TIM_OCInitStructure);
    ...
}
...

```

但是现实是无论我怎么调节，步进电机都不会转(已经确保引脚之间不会冲突)，但是确实有用pwm波控制步进电机的技术，我并未实现。

3.5.2.3 非阻塞式编程

通俗将，便是将每个while循环里总共需要的圈数细分均匀分布到代码中，与数据处理的代码融合，虽然并没有完全解决问题，但是现象改善了许多。

此外，还有一点，这样设置便是将数据处理与转机速度关联，当速度慢，一个while只转一圈，时间短，因此光照强度更新速度也更快，导致不同转速下光照强度更新速度不均匀，为了均衡一下，做了补偿设计，即加一个while的计数变量i，当速度过小时，每当i%50==0，即每间隔50个while循环更新一次光照强度，速度快时则不触发这样虽不能保证更新速度完全一致，但是也大致均衡。

```

#include "system.h"
#include "SysTick.h"
#include "led.h"
#include "usart.h"
#include "adc.h"
#include "lcd.h"
#include "math.h"
#include "stdlib.h"
#include "key.h"
#include "pwm.h"
#include "step.h"
int main()
{
    u8 buf[] = {" illumination "};
    u8 data1[5]={"00200"};
    u8 data2[5];
    u8 danwei[3]={"lux"};
    u8 tool[]={"0123456789"};
    u8 i=0;
    u16 value=0;
    float vol;
    float R0;
    int lux=204;

```

```

int lux_based=204;
int speed=15;
int direction=1;
int delay_tim=2;
int flag=1;
u8 key_value=0;
SysTick_Init(72);
LCD1602_Init();
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
LED_Init();
USART1_Init(9600);
ADCx_Init();
Step_Motor_GPIO_Init();
while(1)
{
    motor_circle(speed,direction,2);
    motor_circle(speed,direction,2);
    key_value=KEY_Scan(0);
    if (key_value==1){
        lux_based=lux;
        key_value=0;
    }
    LCD1602_ShowStr(0, 0, buf);
    LCD1602_ShowStr(0,1,data1);
    motor_circle(speed,direction,2);
    motor_circle(speed,direction,2);
    LCD1602_ShowStr(6,1,data2);
    LCD1602_ShowStr(12,1,danwei);
    i++;
    if (flag==1){
        if(i%50==0){
            motor_circle(speed,direction,2);
            value=Get_ADC_Value(ADC_Channel_3,20);
            motor_circle(speed,direction,2);
            motor_circle(speed,direction,2);
            vol=(float)value*(3.3/4096);
            motor_circle(speed,direction,2);
            R0=10*vol/(3.3-vol);
            motor_circle(speed,direction,2);
            lux=400*pow((10/R0),2);
            motor_circle(speed,direction,2);
            motor_circle(speed,direction,2);
            data1[0]=tool[lux_based/10000];
            data1[1]=tool[(lux_based/1000)%10];
            data1[2]=tool[(lux_based/100)%10];
            data1[3]=tool[(lux_based/10)%10];
            data1[4]=tool[lux_based%10];
            motor_circle(speed,direction,2);
            data2[0]=tool[lux/10000];
            data2[1]=tool[(lux/1000)%10];
            data2[2]=tool[(lux/100)%10];
            data2[3]=tool[(lux/10)%10];
            data2[4]=tool[lux%10];
        }
    }
}

else{

```

```

motor_circle(speed,direction,2);
value=Get_ADC_Value(ADC_Channel_3,20);
motor_circle(speed,direction,2);
motor_circle(speed,direction,2);
vol=(float)value*(3.3/4096);
motor_circle(speed,direction,2);
R0=10*vol/(3.3-vol);
motor_circle(speed,direction,2);
lux=400*pow((10/R0),2);
motor_circle(speed,direction,2);
motor_circle(speed,direction,2);
data1[0]=tool[lux_based/10000];
data1[1]=tool[(lux_based/1000)%10];
data1[2]=tool[(lux_based/100)%10];
data1[3]=tool[(lux_based/10)%10];
data1[4]=tool[lux_based%10];
motor_circle(speed,direction,2);
data2[0]=tool[lux/10000];
data2[1]=tool[(lux/1000)%10];
data2[2]=tool[(lux/100)%10];
data2[3]=tool[(lux/10)%10];
data2[4]=tool[lux%10];
    }
    if (lux-lux_based<30&&lux-lux_based>-30){
        speed=0;
        motor_circle(speed,direction,2);
        motor_circle(speed,direction,2);
    }
    else if (lux<lux_based){
        direction=-1;
        if (lux_based<1030){
            speed=8*(lux_based-lux)/(lux_based-30)+1;
            motor_circle(speed,direction,2);
            motor_circle(speed,direction,2);
        }
        else{
            if(lux>lux_based-1030){
                speed=(lux_based-lux)/1000*8+1;
                motor_circle(speed,direction,2);
                motor_circle(speed,direction,2);}
            else{
                speed=8;
                motor_circle(speed,direction,2);
                motor_circle(speed,direction,2);
            }
        }
    }
    }
    else{
        direction=1;
        if(lux<lux_based+1030){
            speed=(lux-lux_based-30)/1000*8+1;
            motor_circle(speed,direction,2);
            motor_circle(speed,direction,2);}
        else{
            speed=8;
            motor_circle(speed,direction,2);

```

```

        motor_circle(speed,direction,2);
    }
}
if (speed<2){flag=1;}
else{flag=0;}
motor_circle(speed,direction,2);
motor_circle(speed,direction,2);
delay_ms(10);
}
}

```

4 还可以改进的点

4.1 更准确地光照强度与阻值的转换公式

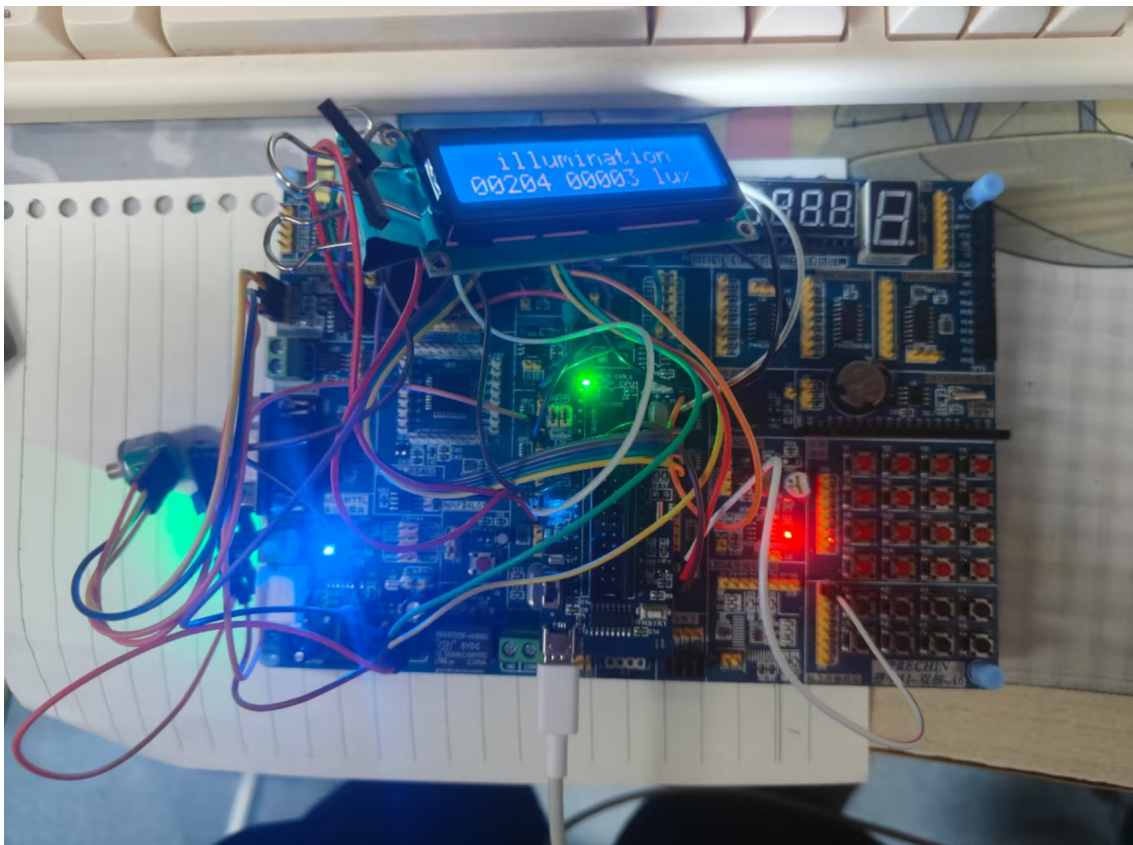
本实验用到的I--R的公式仅在光照强度低的时候大致吻合，且其推导过程与参考值都是估计来的，虽然定性上满足，但是定量上仍然存在较大误差。

4.2 更丝滑的速度过渡与电机

因为main函数中用到了将电机转动分成了各个小块的非阻塞编程，再加上速度是靠整数——圈数来衡量的，这就导致了速度的变化不丝滑，每一个小的speed单位变化，对应的是整个循环过程中总圈数加了12个单位，变化不丝滑。

还有就是上述提到的中断与pwm波方法，应该都能实现电机转动与主程序循环分离，但是受限于实力，没有完成，导致最终成品电机仍存在一点卡顿，能解决卡顿问题将非常完美。

5 现场检查



这是实物图，验收完成后，无改进建议。